

Starlink

Architecture Integration

Payment Gateways | Push Notifications | SMS Gateway | Billing (BSS)

v2.0 | August 2025

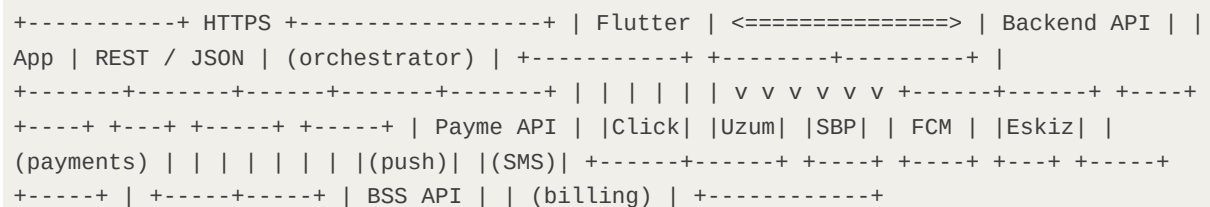
1. Overview

This document describes the architecture for integrating five external services into the Starlink mobile app backend. The Flutter application communicates exclusively with a single backend API, which acts as an orchestrator between the mobile client and external providers: payment gateways (Payme, Click, Uzum), SBP (fast bank transfers), Firebase Cloud Messaging for push notifications, Eskiz SMS gateway for OTP delivery, and the telecom BSS (Business Support System) for billing operations. This design ensures that sensitive credentials, cryptographic keys, and provider-specific logic remain server-side, while the mobile app works with a unified, predictable REST API.

The key architectural principle is that the backend serves as a facade. The Flutter app never communicates directly with Payme, Click, Uzum, Firebase, Eskiz, or the BSS. All integrations are encapsulated within backend service modules, each responsible for a single external provider. This separation allows independent scaling, testing, and replacement of any integration without affecting the mobile client. The backend receives requests via the existing REST API, translates them into provider-specific calls, and returns normalized responses to the app.

1.1 System Diagram

The following diagram illustrates the high-level data flow between the Flutter application, the backend API server, and the five categories of external services. All mobile traffic flows through HTTPS to the backend, which fans out to the appropriate providers. External services call back into the backend via webhooks (payment gateways) or server-to-server APIs (BSS, FCM, Eskiz).



1.2 Technology Stack

Layer	Technology	Role
Mobile	Flutter 3.x / Dart 3.13+	Client application, FCM token management
Backend	Node.js / Express or NestJS	REST API, orchestration, webhook handling
Database	PostgreSQL	Users, transactions, devices, payment sessions
Cache	Redis	OTP codes, rate limits, session state
Queue	Bull / RabbitMQ	Async push delivery, SMS retries
Payments	Payme, Click, Uzum SDK	Payment processing, tokenization

Layer	Technology	Role
Push	Firebase Cloud Messaging	Push notifications to mobile devices
SMS	Eskiz (eskiz.uz)	OTP delivery, service notifications
Billing	BSS REST API	Balance, invoices, service management

2. Payment Gateways

The payment integration supports three major Uzbekistan payment providers: Payme (payme.uz), Click (click.uz), and Uzum Pay (uzum.uz). Each provider offers slightly different capabilities: Payme supports both card payments and SBP transfers, Click focuses on card payments with recurring billing support, and Uzum provides installment payments (6/12/24 months) in addition to standard card processing. The backend abstracts these differences behind a unified payment interface.

2.1 Payment Flow

The payment process follows a redirect-based pattern. When the user initiates a payment in the app, the Flutter client sends a POST `/api/payments/create` request with the amount, payment method, and optional description. The backend creates a payment session in its database, then calls the selected provider API to create a payment transaction. The provider returns a payment URL, which the backend sends back to the app. The app opens this URL in an in-app WebView. After the user completes or cancels the payment, the WebView closes and the app polls POST `/api/payments/{paymentId}/status` until it receives a terminal state (success, failed, or expired).

- 1. App calls POST `/api/payments/create` with {amount, method, description}
- 2. Backend creates payment record in DB (status: pending)
- 3. Backend calls provider API (Payme/Click/Uzum) to create transaction
- 4. Provider returns payment URL
- 5. Backend saves URL, returns {paymentId, paymentUrl, expiresAt} to app
- 6. App opens paymentUrl in WebView
- 7. User completes payment in WebView
- 8. Provider sends webhook to backend (POST `/api/payments/webhook/{provider}`)
- 9. Backend verifies webhook signature, updates DB, registers payment in BSS
- 10. Backend sends push notification to user (payment success/failure)
- 11. App polls GET `/api/payments/{paymentId}/status`, receives terminal state
- 12. App updates UI, shows new balance

2.2 Provider Comparison

Feature	Payme	Click	Uzum
Card payment	Yes	Yes	Yes
SBP transfer	Yes	Yes	No
Installments	No	No	6/12/24 months
Recurring	Yes (subscribe)	Yes (auto-pay)	No
Min amount	1 000 UZS	1 000 UZS	10 000 UZS
Max amount	15 000 000 UZS	15 000 000 UZS	30 000 000 UZS
Webhook	Yes	Yes	Yes
Refund API	Yes	Yes	Yes
Tokenization	Yes (cards)	Yes (cards)	Yes (cards)
Settlement	T+0	T+1	T+1

2.3 Security Considerations

Payment security requires strict separation of concerns. Provider API keys (Payme ID/secret, Click merchant credentials, Uzum keys) are stored in backend environment variables and never exposed to the mobile client. All webhook callbacks are verified using provider-specific signatures: Payme uses SHA-1 HMAC of the request body with the merchant secret, Click uses a preparatory request/response signature exchange, and Uzum uses JWT-based authentication on webhooks. The backend validates the webhook signature before processing any payment status update, preventing spoofed callbacks.

Payment session URLs are single-use and expire after a configurable timeout (default: 30 minutes). The backend assigns each payment a unique paymentId (UUID v4) and stores the mapping between this ID, the provider transaction ID, and the user account. Idempotency is ensured by checking the paymentId on every webhook callback: duplicate notifications for the same paymentId are acknowledged but not reprocessed. All payment operations are logged in an immutable audit table for reconciliation.

3. SBP (Fast Bank Transfers)

The SBP (Fast Payment System) integration in the context of Uzbekistan is implemented through Payme and Click bank transfer capabilities. Unlike card payments, SBP allows users to pay directly from their bank mobile app (Kapitalbank, Uzcard, Humo, etc.) without entering card details. The payment flow is identical to the card payment flow: the app calls POST /api/payments/create with method=sbp, receives a payment URL, and opens it in WebView. The URL redirects to the Payme SBP page where the user selects their bank and confirms the payment in the bank app.

From the backend perspective, SBP does not require a separate integration module. It uses the existing Payme or Click payment module with a different payment type parameter. The provider handles the bank selection and transfer execution. The backend only needs to specify the payment_type as "sbp" when creating the transaction with Payme. Webhook handling and status polling remain identical to card payments. The Transaction entity in the database records the paymentMethod field as "sbp" for filtering and analytics purposes.

SBP Flow Summary: App requests method=sbp -> Backend calls Payme with type=sbp -> Payme returns bank selection URL -> User selects bank in WebView -> Bank app opens -> User confirms -> Webhook to backend -> Push to app

4. Push Notifications (FCM)

Push notifications are delivered through Firebase Cloud Messaging (FCM). The integration requires a Firebase project configured with server key and the google-services.json / GoogleService-Info.plist files added to the Flutter app. On first launch after login, the app obtains an FCM device token and sends it to the backend via POST /api/devices/register. The backend stores this token in the database, linked to the user account. When an event occurs that requires notification (payment success, balance low, service expiring, support reply), the backend sends a push message through the FCM HTTP v1 API.

4.1 Token Lifecycle

FCM device tokens are long-lived but can be invalidated by the user (clearing app data, reinstalling), by Firebase (token refresh), or by the OS (battery optimization). The app must handle token refresh by listening to the onTokenRefresh callback in the Firebase messaging plugin and re-registering the new token with the backend. The backend supports multiple devices per user (phone + tablet), so adding a new token does not invalidate previous ones. On logout, the app calls POST /api/devices/unregister to remove the token, preventing notifications from being sent to a logged-out device.

4.2 Notification Categories

Users can control which notification categories they receive through the notification preferences API (GET/PUT /api/notifications/preferences). Each category can be independently enabled or disabled. When the backend prepares to send a push notification, it checks the user's preferences and suppresses the notification if the category is disabled. This check happens at send time, ensuring real-time preference changes take effect immediately. The following table lists all supported notification categories.

Category	Trigger	Data in Push
paymentSuccess	Payment webhook received (status=success)	amount, method, newBalance

Category	Trigger	Data in Push
paymentFailed	Payment webhook received (status=failed)	amount, errorMessage
balanceLow	BSS reports balance below threshold	amount, paidUntil
serviceExpiring	Service expires within 3 days	serviceName, expiryDate
newsAndPromo	New news article published	title, summary
supportReply	Support ticket receives reply	ticketId, subject

4.3 Flutter Implementation

On the Flutter side, the app uses the `firebase_messaging` package. The `main.dart` initializes Firebase and requests notification permissions. A background message handler is registered for handling notifications when the app is terminated. The foreground handler updates the in-app notification badge count and optionally shows a local notification using the `flutter_local_notifications` package for richer display. The FCM token is obtained via `FirebaseMessaging.instance.getToken()` and sent to the backend immediately after successful login. The `onTokenRefresh` stream listener handles token updates automatically.

5. SMS Gateway (Eskiz)

The SMS integration uses Eskiz (eskiz.uz), a popular SMS aggregation service in Uzbekistan that provides HTTP API access to multiple mobile operators (Uzmobile, Ucell, Beeline, Mobiuz). The primary use case is OTP (One-Time Password) delivery for two-factor authentication. When the user requests a login via SMS, the app calls `POST /api/auth/request-otp` with the phone number. The backend generates a random 6-digit code, stores it in Redis with a 3-minute TTL and a maximum of 3 verification attempts, then sends it via the Eskiz API. The user enters the code in the app, which submits it to `POST /api/auth/verify-otp`. The backend compares the code against the Redis value and returns a JWT token on success.

5.1 Rate Limiting and Anti-Abuse

SMS delivery is rate-limited to prevent abuse and control costs. The backend enforces a per-phone-number rate limit of 1 OTP request per 60 seconds and a maximum of 5 requests per hour. These limits are implemented using Redis counters with TTL. If the rate limit is exceeded, the API returns HTTP 429 with a `retryAfter` field indicating seconds until the next allowed request. The OTP code itself is valid for 3 minutes and allows up to 3 verification attempts. After 3 failed attempts, the code is invalidated and the user must request a new one. All OTP requests and verification attempts are logged for fraud analysis.

5.2 Eskiz API Details

Parameter	Value
Base URL	https://notify.eskiz.uz/api
Auth method	Bearer token (login/email -> token API)
Send endpoint	POST /message/sms/send
Token refresh	POST /auth/refresh (every 20 days)
Batch send	POST /message/sms/send-batch
Callback URL	Configurable per message
Encoding	UTF-8, GSM 7-bit auto-detect

6. Billing System (BSS)

The Business Support System (BSS) is the telecom operator's core system that manages subscriber accounts, service provisioning, rating, billing, and invoicing. The backend API communicates with the BSS via its REST API to perform real-time balance queries, service activation and deactivation, invoice generation, and payment registration. From the Flutter app's perspective, all BSS interactions are transparent: the app calls the usual `/api/account/profile`, `/api/billing/invoice`, or `/api/transactions` endpoints, and the backend translates these into BSS API calls.

6.1 BSS Integration Points

App Endpoint	BSS Operation	Direction
GET /api/account/profile	GetSubscriber + GetBalance + GetServices	Backend -> BSS
GET /api/billing/invoice	GetCurrentInvoice	Backend -> BSS
GET /api/billing/invoice/history	GetInvoiceHistory	Backend -> BSS
POST /api/billing/services/{id}/activate	ActivateService	Backend -> BSS
POST /api/billing/services/{id}/deactivate	DeactivateService	Backend -> BSS
GET /api/transactions	GetTransactionHistory	Backend -> BSS
Webhook /payments/webhook/{provider}	RegisterPayment	Backend -> BSS

6.2 Invoice Management

The BSS generates monthly invoices for each subscriber. The invoice contains a breakdown of all active services with their daily rates, the number of days used in the billing period, the calculated cost

for each service, applicable discounts, and the total amount due. The backend exposes this data through GET /api/billing/invoice (current period) and GET /api/billing/invoice/history (past periods with pagination). Invoice status transitions are managed by the BSS: pending -> paid (when payment webhook is received) -> overdue (after due date). The backend synchronizes invoice status from BSS on each request to ensure consistency.

6.3 Service Management

Service activation and deactivation are requested through dedicated endpoints (POST /api/billing/services/{id}/activate and /deactivate). The backend forwards these requests to the BSS, which validates the request (e.g., checking if the service is compatible with the current tariff, if there are no outstanding debts, or if the service is already active). The BSS responds with success or a specific error code. The backend maps BSS error codes to HTTP status codes: 200 for success, 409 for conflicts (already active), and 422 for validation errors. Service changes take effect immediately in the BSS and are reflected in the next GET /api/account/profile call.

7. New API Endpoints Summary

The following table summarizes all new endpoints added to the API specification (api.yaml v2.0) to support the five external service integrations. Existing endpoints (/auth/login, /account/profile, /transactions, /news, /support) remain unchanged in their request/response format, though some now have enriched behavior (e.g., transactions now include paymentMethod and providerTransactionId fields).

Method	Endpoint	Purpose	Ext. Service
POST	/auth/request-otp	Send SMS with OTP code	Eskiz
POST	/auth/verify-otp	Verify OTP, return JWT	Eskiz
POST	/payments/create	Create payment session	Payme/Click/Uzum
GET	/payments/{id}/status	Poll payment status	Payme/Click/Uzum
POST	/payments/webhook/{p}	Provider callback	Payme/Click/Uzum
POST	/devices/register	Register FCM token	Firebase
POST	/devices/unregister	Remove FCM token	Firebase
GET	/notifications/preferences	Get push settings	Firebase
PUT	/notifications/preferences	Update push settings	Firebase
GET	/billing/invoice	Current invoice from BSS	BSS
GET	/billing/invoice/history	Past invoices	BSS
POST	/billing/services/{id}/activate	Activate service	BSS

Method	Endpoint	Purpose	Ext. Service
POST	/billing/services/{id}/deactivate	Deactivate service	BSS

8. Backend Module Structure

The backend should be organized into dedicated service modules, each encapsulating a single external integration. This modular architecture allows independent development, testing, and deployment of each integration. Each module exposes a consistent interface (create, getStatus, handleWebhook) that the API routes layer delegates to. The following structure is recommended for the Node.js/NestJS backend implementation.

```
src/
modules/
auth/ # login, OTP request/verify
payment/
payment.service.ts # unified payment interface
providers/
payme.service.ts # Payme API wrapper
click.service.ts # Click API wrapper
uzum.service.ts # Uzum API wrapper
payment.controller.ts
payment.webhook.controller.ts
push/
fcm.service.ts # Firebase Cloud Messaging
device.controller.ts
sms/
eskiz.service.ts # Eskiz SMS gateway
billing/
bss.service.ts # BSS API wrapper
invoice.controller.ts
service.controller.ts
account/ # existing profile logic
transaction/ # existing transaction logic
common/
config/ # env vars, provider credentials
database/ # PostgreSQL, TypeORM/Prisma
redis/ # OTP cache, rate limits
queue/ # Bull job queue
```

9. Implementation Priority

The recommended implementation order follows the dependency graph and business value. Payment integration should be implemented first since it directly enables revenue collection. Push notifications come second as they enhance the payment experience (users receive instant confirmation) and improve engagement. SMS/OTP is third, adding a second authentication factor. Billing integration is fourth, providing real invoice data instead of mock values. SBP is last because it builds on top of the

existing Payme integration with minimal additional effort.

Priority	Integration	Effort	Dependencies
P0 (Week 1-2)	Payme payment gateway	High	BSS payment registration
P0 (Week 2-3)	Click payment gateway	Medium	Payment module from P0
P1 (Week 3-4)	FCM push notifications	Medium	Firebase project setup
P1 (Week 4-5)	Eskiz SMS / OTP	Low	Redis for rate limiting
P2 (Week 5-6)	BSS billing integration	High	BSS API access, data mapping
P2 (Week 6)	Uzum payment gateway	Medium	Payment module from P0
P3 (Week 6)	SBP via Payme	Low	Payme module from P0